

Nkentseu

Un moteur logiciel complet, écrit intégralement à partir de zéro

Prix d'excellence 2026 - 35 jeunes de moins de 35 ans du Cameroun

Catégorie : **Technologie et innovation**

TEUGUIA TADJUIDJE Rodolf Séderis

Ingénieur polytechnicien • Enseignant à l'ENSP Yaoundé • CEO de Rihen (10 personnes)
Yaoundé, Région du Centre, Cameroun

github.com/Rihen-Universe/Nkentseu

linkedin.com/in/rodolf-séderis-teugua-tadjuidje

En une phrase. Un moteur logiciel de plus de vingt-cinq modules, portable sur huit plateformes, dont chaque brique - du décodeur vidéo au moteur de rendu 3D, du système d'intelligence artificielle au compilateur de shaders - a été écrite à la main, sans aucune bibliothèque tierce, par une seule personne, depuis le Cameroun.

Ce que ce dossier prouve

200+	étudiants formés à l'ENSP Yaoundé depuis février 2024
10	personnes dans la structure Rihen qu'il dirige
815	commits sur le dépôt public, 4 branches
25+	modules écrits from-scratch
8	plateformes cibles (Windows, Linux, macOS, Android, iOS, Web, HarmonyOS, Xbox)
6	pilotes graphiques (Vulkan, OpenGL, DirectX 11, DirectX 12, Metal, logiciel)
82,6 %	du code en C++, écrit à la main
16	forks, 11 étoiles, 2 contributeurs
13	rapports de bugs ouverts par des <i>utilisateurs tiers</i>
501	projets compilables déclarés par l'espace de travail
196	pages de manuel technique rédigé en français
100 017	paires de dialogues constituées pour l'entraînement d'une IA

Toutes ces valeurs sont visibles sur les captures qui suivent, ou vérifiables publiquement sur le dépôt. Ce dossier ne contient aucune estimation.

Le moteur de rendu - écrit sans aucune bibliothèque graphique

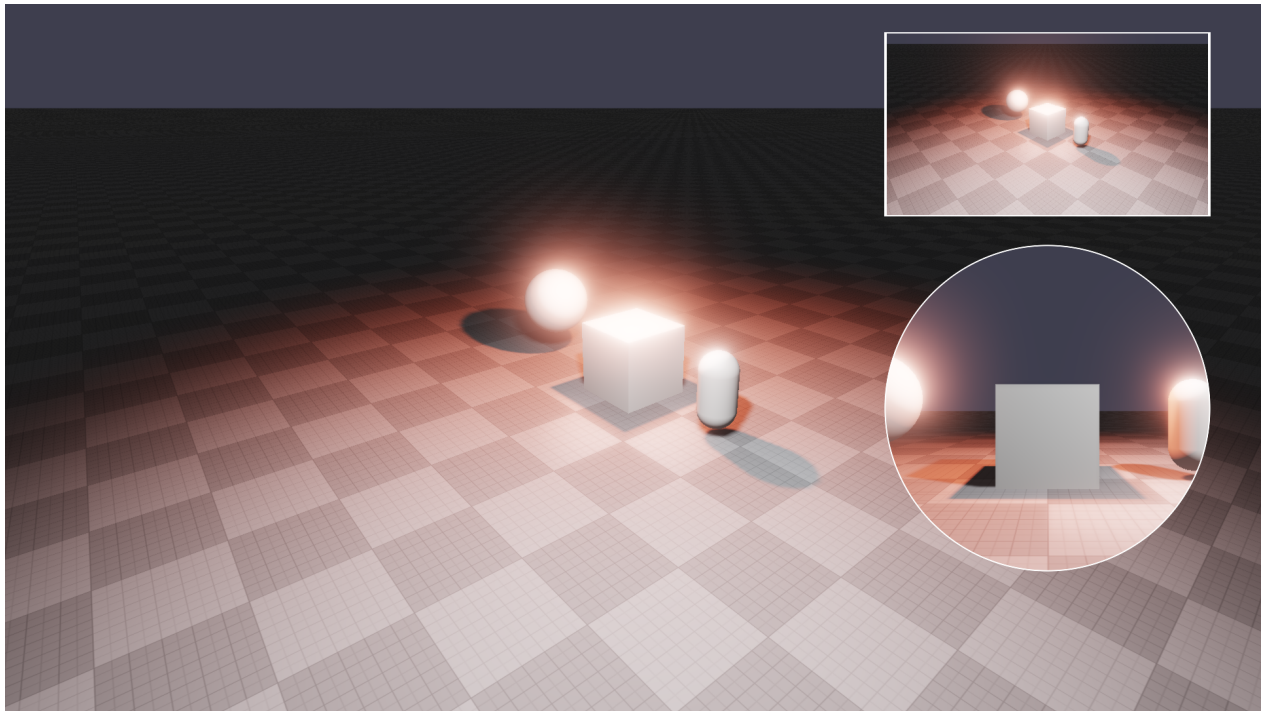


FIG. 1 – Image produite par le moteur Nkentsu. Éclairage physique (PBR), ombres douces par cartes d'ombres virtualisées, halo lumineux, occlusion ambiante, correction colorimétrique cinéma (ACES). Aucune bibliothèque de rendu tierce n'intervient : le pipeline complet, du calcul de la lumière jusqu'au pixel, est écrit dans le projet. Les vignettes en haut et à droite sont des incrustations générées par le système de sortie du logiciel.

Ce niveau de rendu - matériaux physiquement corrects, ombres, réflexions, post-traitement - représente environ 80 % du socle d'un moteur professionnel. Il fonctionne à l'identique sur six pilotes graphiques différents, grâce à **NkSL**, un langage de shaders conçu pour ce projet : une seule écriture, traduite automatiquement vers les cinq API du marché.

NK3DModeler - le logiciel de modélisation 3D bâti sur le moteur

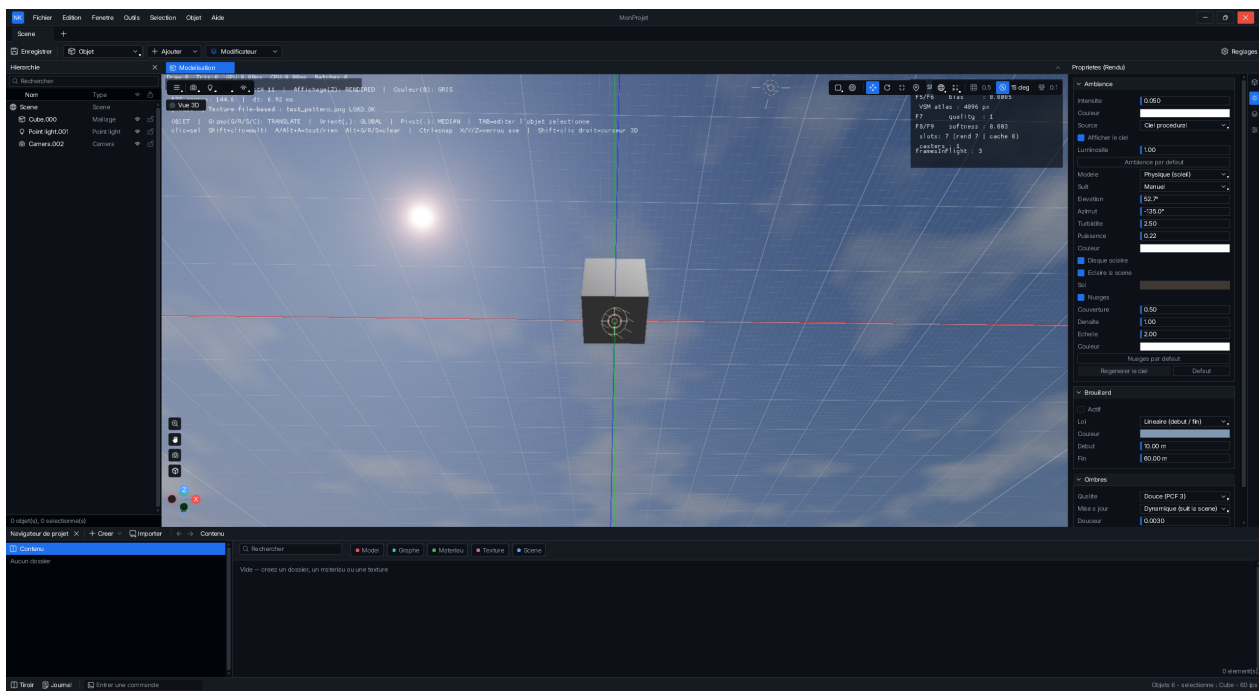


FIG. 2 – NK3DModeler : interface complète de modélisation 3D construite sur le moteur. Hiérarchie de scène, panneaux de propriétés, gizmos de transformation, aimantation géométrique, système de sortie (rendu d'images et enregistrement vidéo). Toute l'interface est dessinée par le moteur lui-même.

NKCode - l'environnement de développement, également écrit from-scratch

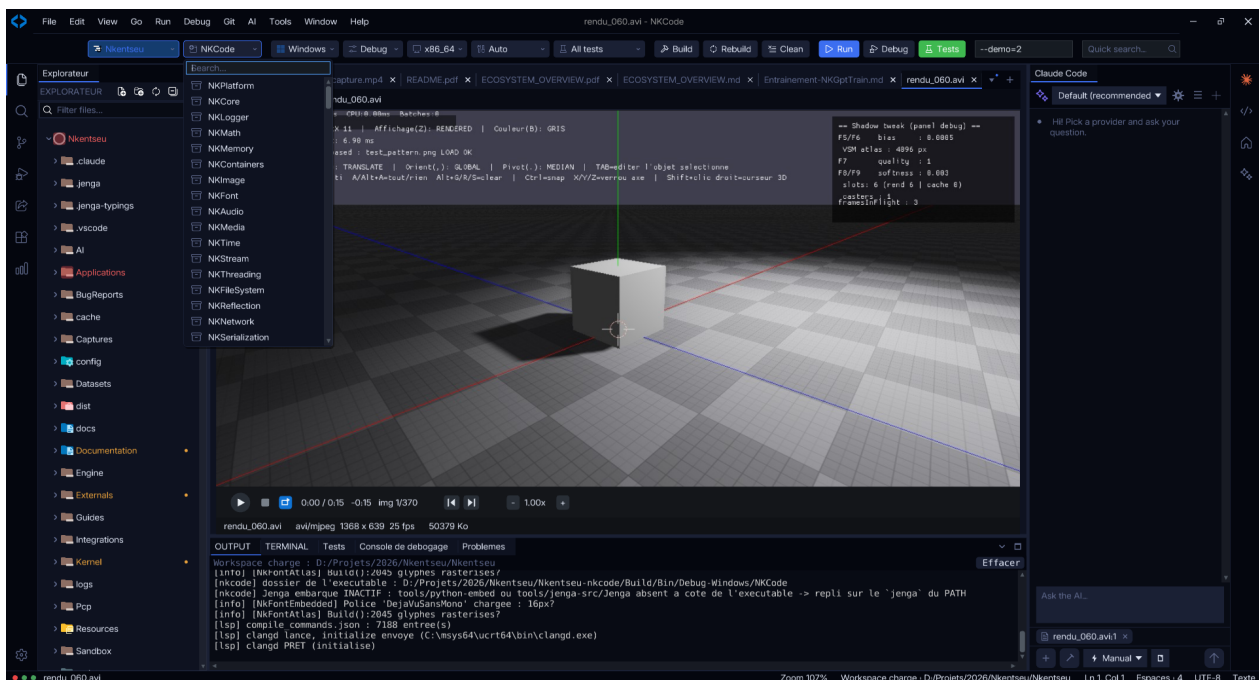


FIG. 3 – NKCode lisant une vidéo produite par le moteur (format AVI/MJPEG, 1368×639, 25 images/s) - le lecteur vidéo, le décodeur et le fichier proviennent tous du même projet. À gauche, la liste des modules du moteur : NKPlatform, NKCore, NKLogger, NKMath, NKMemory, NKContainers, NKImage, NKFont, NKAudio, NKMedia, NKTime, NKStream, NKThreading, NKFileSystem, NKReflection, NKNetwork, NKSerialization. En bas, la compilation en cours.

Cette capture résume à elle seule la nature du projet : *un environnement de développement maison, compilant un moteur maison, lisant une vidéo produite par ce moteur et décodée par son propre décodeur.*

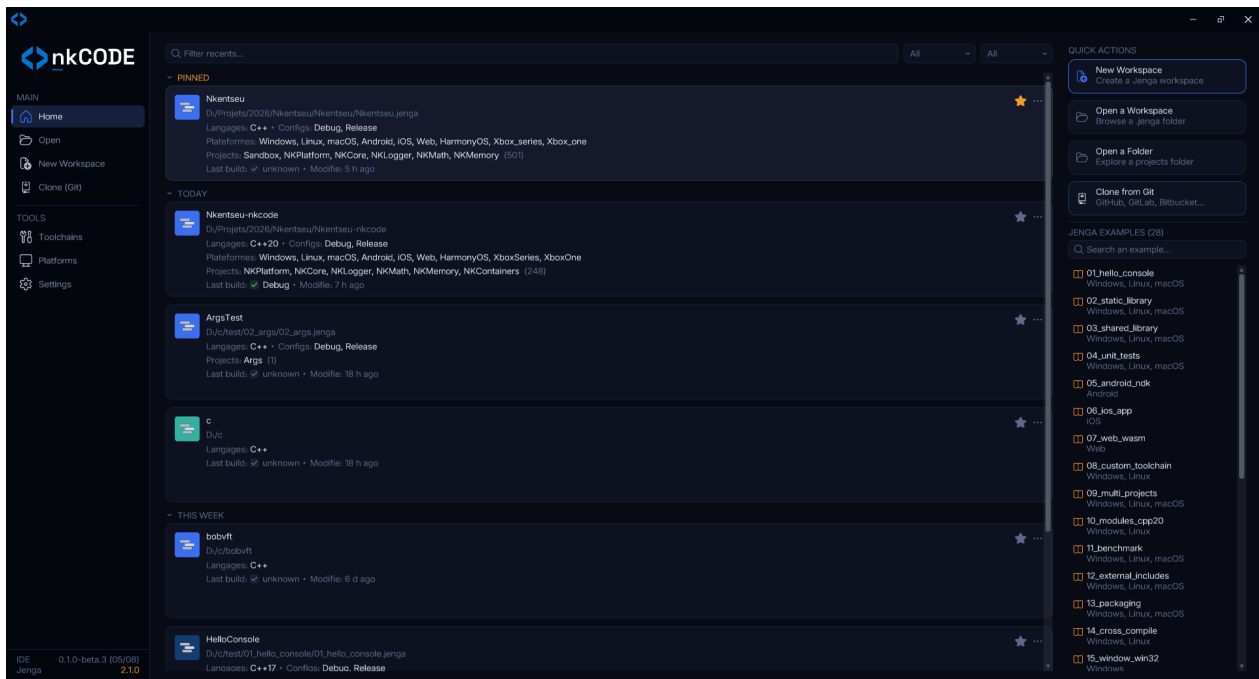


FIG. 4 – Écran d'accueil de NKCode. L'espace de travail Nkentsau y déclare **501 projets** compilables et **8 plateformes** cibles. À droite, 28 exemples fournis avec Jenga, le système de compilation également développé pour ce projet. En bas à gauche : les versions réelles des deux outils.

Cet écran révèle une pièce que le reste du dossier ne montre pas : **Jenga**, le *système de compilation* conçu pour ce projet. Construire un moteur portable sur huit plateformes suppose un outil capable de décrire, configurer et compiler des centaines de cibles pour chacune d'elles. Aucun système existant ne répondait au besoin sans imposer ses propres dépendances : Jenga a donc été écrit aussi. Il gère ici **501 projets**, huit plateformes et deux configurations, et il est livré avec 28 exemples pédagogiques. C'est l'outil qui rend le reste possible - et le seul élément du dossier dont l'absence aurait rendu tout le reste irréalisable.

Le dépôt public - activité et adoption

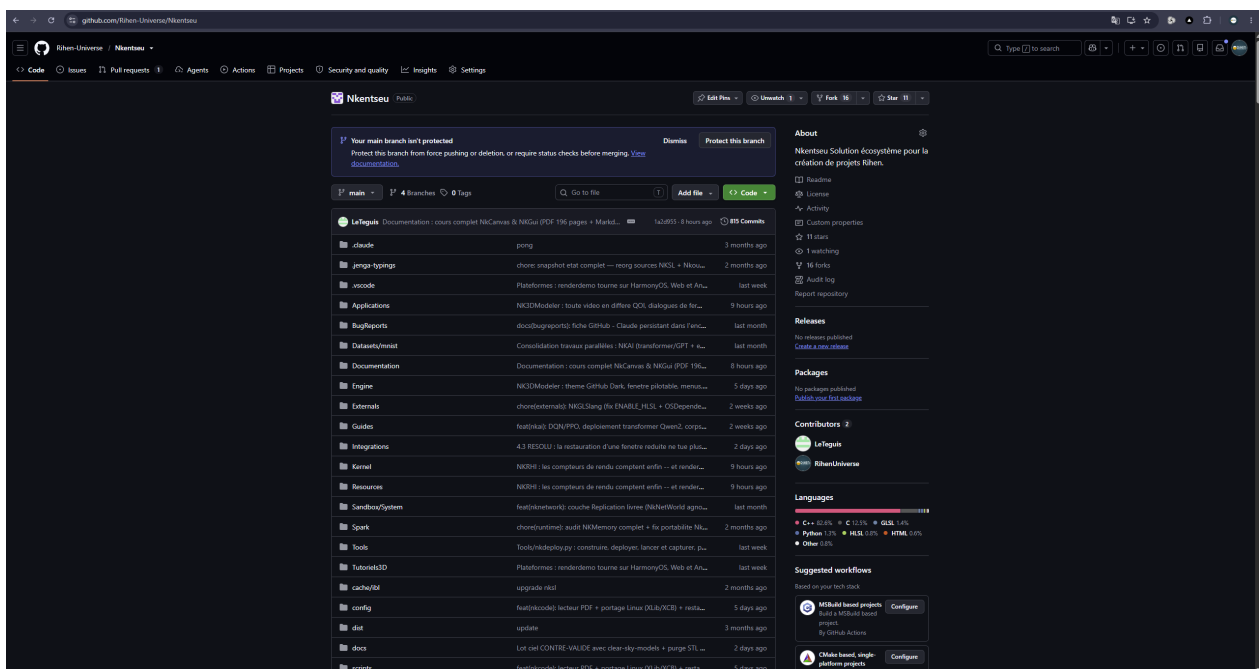


FIG. 5 – Dépôt public `github.com/Rihen-Universe/Nkentseu` : 815 commits, 4 branches, 16 forks, 11 étoiles, 2 contributeurs. Répartition des langages : C++ 82,6 %, C 12,5 %, GLSL 1,4 %, Python 1,3 %, HLSL 0,8 %. Les messages de commit, rédigés en français, décrivent des travaux d'ingénierie précis et datés.

Preuve d'usage par des tiers

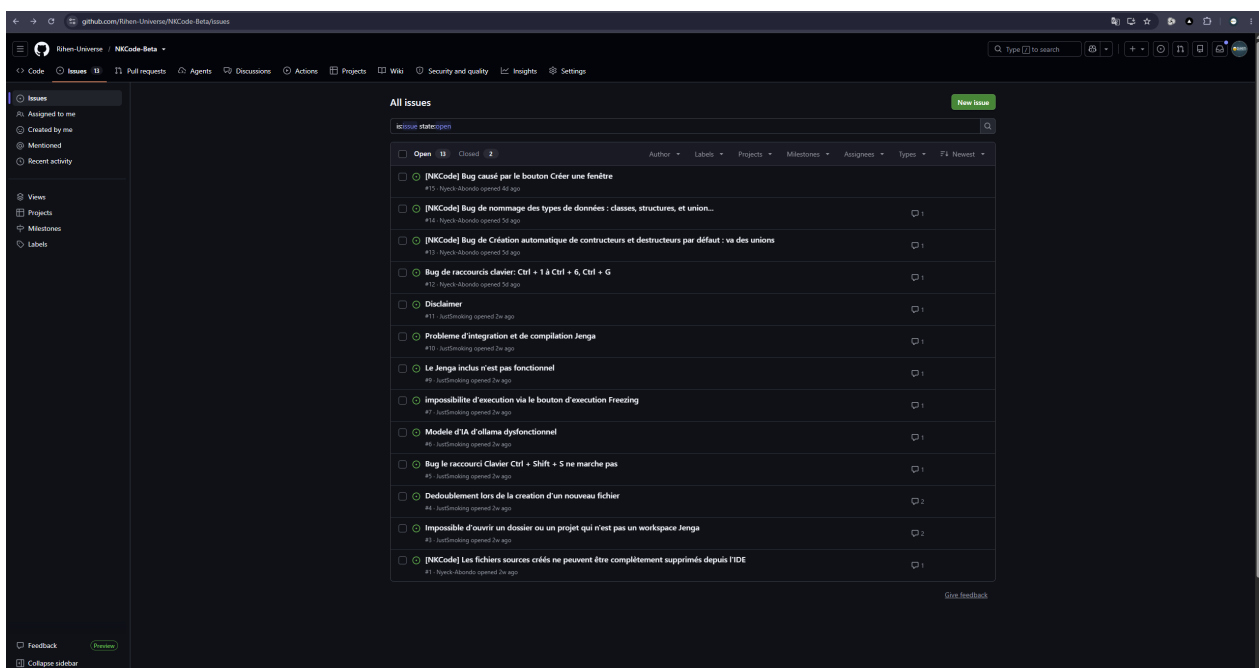


FIG. 6 – Rapports de bugs ouverts sur NKCode-Beta par des utilisateurs *extérieurs au projet* (comptes Nyeck-Abondo et JustSmoking) : 13 tickets ouverts, 2 résolus. Ils portent sur l'usage réel du logiciel - raccourcis clavier, création de fenêtres, intégration du système de compilation, ouverture de dossiers.

Cette page établit que le travail est **utilisé, testé et discuté** par d'autres personnes que son auteur. Chaque ticket représente quelqu'un qui a installé le logiciel, s'en est servi assez longtemps pour rencontrer une limite, et a pris le temps de la décrire. Les 16 forks du dépôt vont dans le même sens.

La transmission, à l'école et par le code

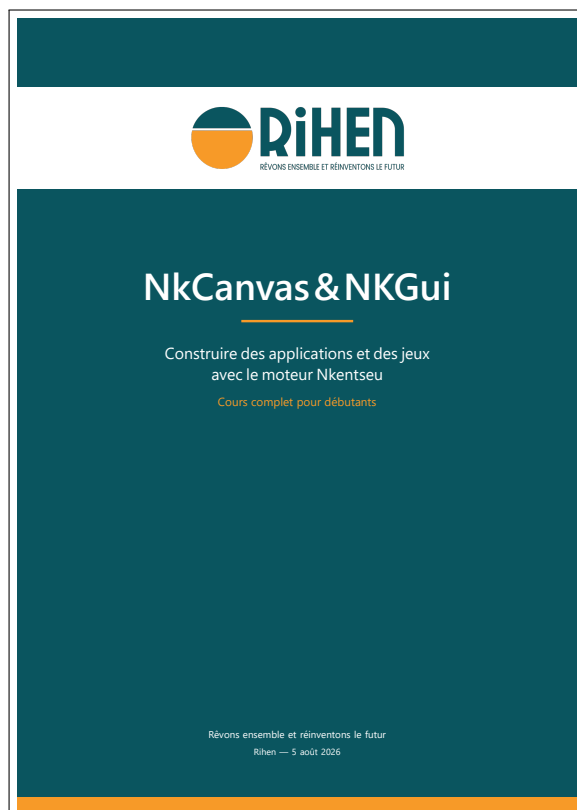
L'auteur est **enseignant à l'École Nationale Supérieure Polytechnique de Yaoundé** depuis **février 2024**, où il a déjà formé **plus de deux cents étudiants** - soit une centaine par an. Ce projet n'est donc pas un travail solitaire refermé sur lui-même : il alimente directement la formation d'ingénieurs camerounais, et leur montre sur un cas réel ce que signifie construire une infrastructure logicielle plutôt que l'assembler. Peu d'enseignants peuvent ouvrir devant leurs étudiants le code complet d'un moteur de rendu, d'un décodeur vidéo ou d'un compilateur de shaders, et expliquer chaque ligne parce qu'ils l'ont écrite.

C'est là que le manuel de 196 pages prend son sens : il prolonge le cours au-delà de la salle, en français, pour ceux qui voudront aller plus loin que ce qu'un semestre permet d'enseigner.

Il est également **fondateur et dirigeant de RiHEN**, structure de **dix personnes** sous laquelle l'écosystème est publié - ce qui inscrit le travail dans une perspective économique et collective, et non dans le seul exercice technique.

Une précision qui compte pour la lecture de ce dossier : **Nkentseu et Jenga sont l'œuvre d'une seule personne**. Les dix membres de la structure travaillent sur d'autres activités ; l'écosystème présenté ici - moteur, système de compilation, environnement de développement, modelleur 3D, sous-système d'intelligence artificielle - a été conçu et écrit intégralement par l'auteur. Diriger une équipe de dix personnes tout en portant seul une infrastructure logicielle de cette ampleur relève de deux compétences distinctes, rarement réunies.

La transmission - un manuel de 196 pages



Un cours complet de **196 pages**, rédigé en français, enseigne à un débutant comment construire des applications avec le moteur : les modules de base, le rendu 2D, le système d'interface, la construction d'une application complète, puis l'intégration des images, des polices, de l'audio, de la vidéo et du réseau.

Le document contient environ **260 extraits de code**, chacun accompagné du fichier et de la ligne exacte dont il provient - afin que le lecteur puisse vérifier. Il comporte 39 exercices, et signale explicitement ce qui ne fonctionne pas encore dans le moteur.

Cette exigence - dire ce qui marche *et* ce qui ne marche pas - traverse tout le projet. Elle est ce qui rend vérifiable chacune des affirmations de ce dossier.

L'intelligence artificielle et la langue ghomala

Nkentseu comprend **NKAI**, un sous-système d'intelligence artificielle écrit lui aussi entièrement à la main : calcul tensoriel sur processeur et carte graphique, différentiation automatique, réseaux de neurones, apprentissage par renforcement, modèles génératifs. Résultats mesurés : **96,3 %** de précision en reconnaissance de chiffres manuscrits, génération de maillages 3D à partir de bruit, et -

travaux d'août 2026 - exécution d'un modèle de langage de **7 milliards de paramètres sur une carte graphique de 8 Go**, grâce à des noyaux de calcul quantifiés atteignant **294 milliards d'opérations par seconde**.

Un corpus de **100 017 paires de dialogues** en français a été constitué pour l'entraînement, incluant la traduction vers le **ghomala**, langue camerounaise. Les langues africaines sont massivement absentes des technologies du langage : ce qui n'est pas outillé numériquement disparaît. Ce volet du projet vise directement ce manque.